Programación de Entornos Multimedia

SEMANA 4 — SESIÓN EN VIVO · 3 HORAS

Campo	Detalle
Asignatura	Programación de Entornos Multimedia
Ciclo	Momento 1 · Fundamentos 3D e Interactividad Básica
Semana	Semana 4 — Sesión en Vivo de 3 Horas
Docente	**Mg. Gabriel Guerrero Arellano**
Institución	Universidad Cooperativa de Colombia

Tema Principal

Iluminación Dinámica, Sombras Realistas PBR y Sistemas de Partículas Masivos

> *"En WebGL, la geometría define la estructura, pero la física de la luz y las partículas crean la atmósfera y la inmersión."*

Hilo Conductor — Resumen de las Semanas 1 a 3

Cada semana ha construido una capa de complejidad sobre la anterior. Antes de entrar al fotorrealismo de hoy, revisamos el camino recorrido:

1

Semana 1 — Fundamentos 3D y Pipeline WebGL

Entendimos el flujo gráfico completo: **Escena (Scene)**, **Cámara en perspectiva (PerspectiveCamera)** y **Lienzo de dibujo (WebGLRenderer)**. Desplegamos el entorno de desarrollo ultrarrápido con **Node.js y Vite**.

2

Semana 2 — Geometrías, Materiales y Transformaciones

Creación de mallas (Meshes) combinando formas primitivas (cubos, esferas) con **MeshBasicMaterial**. Manipulación espacial en 3 ejes: **Posición, Rotación y Escala (TRS: Translate, Rotate, Scale)**.

3

Semana 3 — Animación, Video y Audio Espacial

Construcción del bucle de renderizado continuo a **60 cuadros por segundo** mediante `requestAnimationFrame`. Integración de texturas de video dinámicas y sonido tridimensional posicional.

Hoy damos el salto cualitativo: de materiales planos a física de luz real, sombras computadas y simulación masiva de partículas.

Conexión Inmediata — Lo Visto el Pasado Miércoles (Semana 3)

¿Qué logramos el miércoles pasado?

- **Sincronización Multimedia**

  Incrustamos un elemento `<video>` de HTML5 dentro de una malla 3D mediante `THREE.VideoTexture`, logrando superficies con contenido audiovisual en tiempo real.

- **Audio Espacial 3D**

  Implementamos `THREE.PositionalAudio` acoplado a un `THREE.AudioListener` en la cámara, experimentando cómo el volumen y el paneo cambian según la distancia del usuario.

¿Por qué hoy cambia el paradigma?

→ Todo lo que construimos hasta el miércoles usaba **materiales planos o no iluminados**.

→ Si apagábamos las luces ficticias, los objetos **se seguían viendo igual** — sin reacción a la luz.

→ Hoy damos el salto al **fotorrealismo**: aprenderemos cómo la luz real rebota en los objetos (PBR) y cómo llenar el espacio con miles de partículas en movimiento.

Fase Conceptual 1 — El Modelo PBR (Physically Based Rendering)

¿Qué es PBR?

Es el **estándar moderno de la industria** (usado en Unreal Engine, Blender y Three.js) para simular cómo los fotones interactúan con la materia física. No es una aproximación artística: es una simulación basada en las leyes de la óptica.

Principio de Conservación de Energía

Un objeto **jamás puede reflejar más luz de la que recibe**. La luz que impacta una superficie se divide en dos caminos: **luz reflejada especular** (brillo) y **luz absorbida/refractada** (color difuso).

Eliminación del "Efecto Plástico"

Los modelos clásicos antiguos (como **Phong o Lambert**) producían brillos artificiales; PBR garantiza materiales naturales bajo cualquier condición de luz.

Adopción en la Industria

Unreal Engine

Motor AAA de videojuegos

Blender

Render Cycles/EEVEE

Three.js

WebGL en navegador

Unity HDRP

Pipeline de alta definición

Parámetros PBR Esenciales — Roughness vs Metalness

1. Roughness (Rugosidad)

Modela las **micro-imperfecciones de la superficie** a nivel microscópico. Controla cómo se dispersan los rayos de luz al rebotar.

Valor 0.0 — Espejo

Superficie perfectamente pulida. Reflejos nítidos y definidos. Ejemplo: espejo, agua en calma, metal pulido.

Valor 1.0 — Mate

Superficie rugosa o porosa. Dispersa la luz en todas direcciones. Ejemplo: madera rústica, tiza, yeso.

2. Metalness (Metalicidad)

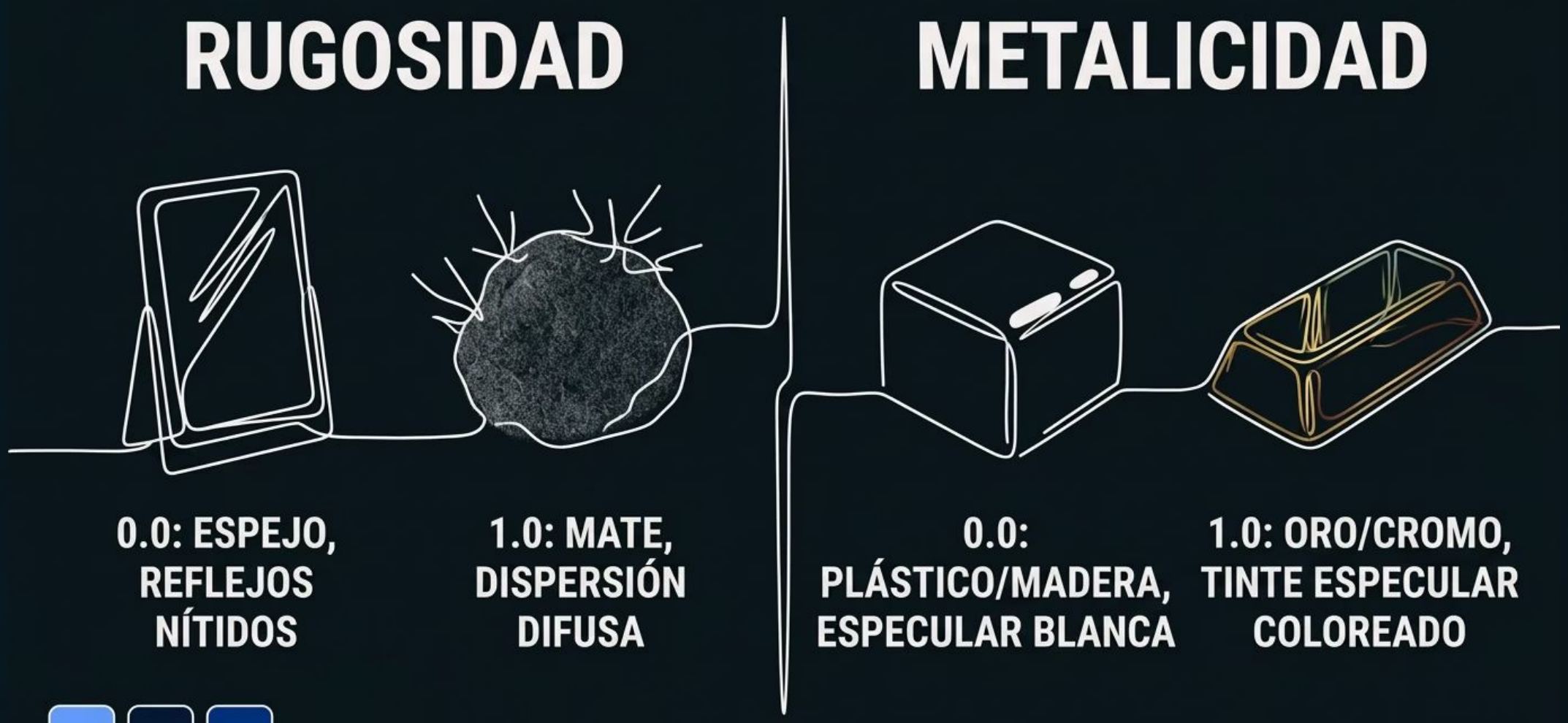
Distingue materiales **dieléctricos (aislantes)** de **conductores eléctricos**. Determina cómo el material interactúa con la componente especular de la luz.

Dieléctricos (0.0)

Plásticos, maderas, telas. Tienen reflejos monocromáticos (blancos). La luz difusa domina el color percibido.

Metales (1.0)

Oro, cromo, cobre. Absorben toda la luz difusa y **tiñen el reflejo** con el color del propio metal.



Materiales Físicos Avanzados — MeshPhysicalMaterial

Three.js ofrece `MeshStandardMaterial` para PBR básico, pero **MeshPhysicalMaterial** añade propiedades fotorrealistas de alta gama que simulan fenómenos ópticos complejos:

Clearcoat (Barniz)

Simula una **segunda capa de laca transparente brillante** por encima de la superficie base. Ideal para pintura de automóviles o cerámica esmaltada. Parámetros: `clearcoat: 1.0`, `clearcoatRoughness: 0.1`.

Transmission (Transmisión)

Permite que los **rayos atraviesen el material** simulando vidrio o cristales ópticos reales sin romper el cálculo de sombras. Esencial para ventanas, botellas y lentes.

Sheen (Terciopelo)

Simula la **retrodispersión de la luz en los bordes** de telas finas y terciopelo. Produce el característico halo luminoso en los contornos de materiales textiles.

MeshPhysicalMaterial tiene un costo computacional mayor que MeshStandardMaterial. Úsalo selectivamente en los objetos protagonistas de la escena.

El Catálogo de Luces en Three.js

En WebGL **no existe la luz por defecto**; la oscuridad es absoluta hasta que instanciamos emisores explícitamente. Cada tipo de luz tiene un costo computacional y un caso de uso específico:

THREE.AmbientLight

Luz ambiental omnidireccional. Baña todos los polígonos por igual; **no tiene dirección ni genera sombras**. Costo GPU: mínimo.

THREE.DirectionalLight

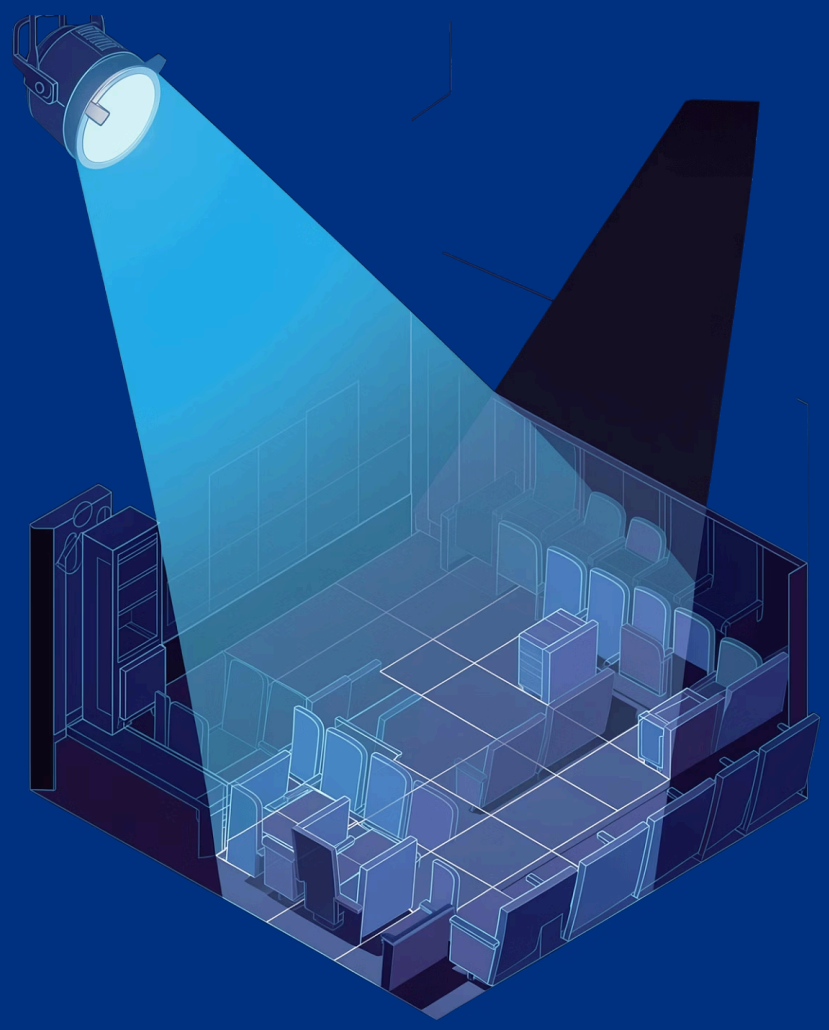
Simula el Sol. Emite **rayos paralelos desde el infinito** y cubre grandes extensiones. Costo GPU: bajo-moderado.

THREE.PointLight

Bombilla puntual. Emite luz en **360 grados esféricos** y su potencia cae con la distancia. Costo GPU: alto con sombras.

THREE.SpotLight

Reflector de teatro. Proyecta un **cono delimitado** por ángulo, atenuación y penumbra. Costo GPU: moderado y eficiente.



Iluminación de Escena — Relleno vs Contraste

La Regla de Oro Lumínica

> ⚠ Nunca uses AmbientLight al 100%: Si subes mucho la luz ambiental, la escena pierde profundidad, se aplana y parece un dibujo sin volumen.

La luz ambiental existe únicamente para evitar que las sombras sean negras absolutas. No debe ser la fuente de iluminación principal.

Configuración Recomendada para Escenas PBR

1 Luz Ambiental Baja

Mantén la intensidad entre **0.1 y 0.3** solo para que las sombras no queden negras al 100%. Actúa como relleno de sombras.

2 Key Light Principal

Utiliza una luz direccional o focal como **luz principal (Key Light)** para crear volumen, contraste y sombras proyectadas. Esta es la que define la atmósfera.



Fuentes Focales — PointLight vs SpotLight

PointLight — Fuego / Velas / Focos

Emisión

Emite en **todas las direcciones** simultáneamente (360° esférico).

Costo en GPU: MUY ALTO

Si proyecta sombras, debe generar un **mapa de sombras cúbico de 6 caras** (arriba, abajo, izquierda, derecha, adelante, atrás) — 6 renders por cuadro.

Casos de Uso

Antorchas, velas, explosiones, efectos de fuego localizados.

SpotLight — Linternas / Faros / Cañones de Luz

Emisión

Emite en un **ángulo cónico configurable** mediante los parámetros `angle` y `penumbra`.

Costo en GPU: MODERADO Y EFICIENTE

Solo requiere un **único plano de proyección** para calcular la sombra — 1 render adicional por cuadro.

Casos de Uso

Faros de automóvil, reflectores de escenario, linternas, iluminación arquitectónica.

La Mecánica Computacional del Shadow Map

¿Cómo sabe la tarjeta gráfica dónde hay sombra? El proceso ocurre en **tres pasos secuenciales por cada cuadro renderizado**:



Paso 1 — Pase de la Luz

La GPU renderiza la escena internamente colocándose en la posición del foco de luz y guarda un **mapa de distancias en blanco y negro** llamado Shadow Map.

Paso 2 — Pase de la Cámara

La GPU dibuja la escena desde los **ojos del usuario**, usando la geometría y materiales normales de la escena.

Paso 3 — Comparación Z

Para cada fragmento, compara si la distancia hacia la luz es mayor que la guardada en el mapa. Si hay un objeto intermedio, el fragmento se **oscurece**.

Algoritmos de Filtrado y Suavizado de Sombras

Three.js permite configurar la calidad del renderizador mediante `renderer.shadowMap.type`. La elección impacta directamente en el realismo visual y el costo computacional:

1

THREE.BasicShadowMap

Sin filtrado. Es el más rápido, pero los bordes de la sombra se ven pixelados y dentados. Útil solo para prototipos o hardware muy limitado.

2

THREE.PCFShadowMap

Percentage-Closer Filtering. Suaviza los bordes tomando muestras intermedias. Calidad estándar para la mayoría de proyectos en producción.

3

THREE.PCFSoftShadowMap

Algoritmo avanzado que dispersa las muestras para generar penumbras suaves y realistas. Ideal para la clase de hoy. Mayor costo, máxima calidad visual.

Configura siempre el tipo de shadow map antes de añadir luces a la escena. Cambiar el tipo en tiempo de ejecución puede causar artefactos visuales.

Sistemas de Partículas — El Cuello de Botella de la CPU

El Problema del Dibujo Masivo

Enfoque Ingenuo: 3,000 Meshes

Si instanciamos 3,000 mallas cúbicas individuales (`THREE.Mesh`), la CPU deberá emitir **3,000 llamadas de dibujo (Draw Calls)** por cuadro.

Consecuencia

Esto **colapsa cualquier navegador**, provocando caídas de FPS de 60 a 10 cuadros. La CPU se convierte en el cuello de botella.

La Solución Arquitectónica: THREE.Points

- Agrupamos los 3,000 puntos en una **única geometría de búfer**.
- La CPU hace **1 solo llamado de dibujo (1 Draw Call)** independientemente del número de partículas.
- Le delega el **cálculo espacial a los procesadores paralelos de la GPU**, que están diseñados para este tipo de operaciones masivas.

3,000	1	60
Draw Calls (Meshes)	Draw Call (Points)	FPS Objetivo
Enfoque ingenuo — colapsa el navegador	Solución arquitectónica — 60 FPS estables	Rendimiento fluido con THREE.Points

Geometría de Búfer (THREE.BufferGeometry)

Para manejar miles de partículas eficientemente en memoria, Three.js utiliza arreglos binarios planos que se transfieren directamente a la GPU:

1 Arreglo Binario Plano

Creamos un arreglo binario de precisión simple: `new Float32Array(cantidad * 3)`. El factor 3 corresponde a los componentes X, Y, Z de cada partícula.

2 Memoria Continua (Cache-Friendly)

Los datos se guardan secuencialmente en memoria continua: `[x0, y0, z0, x1, y1, z1, x2, y2, z2, ...]`. Esta disposición es óptima para la transferencia a la GPU.

3 Asignación del Atributo

Asignamos el arreglo al atributo de posición de la geometría para que Three.js lo envíe al shader de vértices.

Código Clave

```
// Crear arreglo de posiciones
const cantidad = 3000;
const posiciones = new Float32Array(
 cantidad * 3
);

// Llenar con coordenadas aleatorias
for (let i = 0; i < cantidad; i++) {
 posiciones[i * 3] = x; // X
 posiciones[i * 3 + 1] = y; // Y
 posiciones[i * 3 + 2] = z; // Z
}

// Asignar a la geometría
particulasGeo.setAttribute(
 'position',
 new THREE.BufferAttribute(
 posiciones, 3
 )
);
```

Materiales de Partículas y Mezcla Aditiva

La apariencia visual de las partículas depende de tres configuraciones críticas del material que trabajan en conjunto:

THREE.PointsMaterial

Controla el **tamaño de cada punto** (`size`) y permite que se hagan pequeños con la distancia (`sizeAttenuation: true`). Simula la perspectiva natural de las partículas en el espacio 3D.

THREE.AdditiveBlending

Mezcla Aditiva: En lugar de tapar lo que hay detrás, los píxeles superpuestos **suman sus valores de color**. Es el truco óptico fundamental para lograr fuegos incandescentes, chispas brillantes, rayos mágicos o rayos láser.

depthWrite: false

Desactiva la escritura de profundidad para **evitar que partículas transparentes proyecten bordes negros no deseados** sobre otras partículas. Esencial cuando se usa transparencia con blending aditivo.

La combinación de AdditiveBlending + depthWrite: false + transparent: true es la receta estándar de la industria para efectos de partículas luminosas.

Prerrequisitos y Setup del Laboratorio (Anti-Errores)

Chequeo Previo — Evitar Fallos Técnicos

Node.js

Versión **18+ o 20+ LTS** instalada. Verificar con: `node -v`

Navegador

Chrome o Edge con aceleración gráfica por hardware activada en la configuración del sistema.

WebGL Activo

Verificar que el navegador soporte **WebGL 2.0** ingresando a `chrome://gpu`.

Inicialización del Entorno en la Terminal

```
# Crear proyecto con Vite
npm create vite@latest taller-semana4 \
  -- --template vanilla

# En UNA NUEVA TERMINAL Entrar al directorio
cd taller-semana4

# Instalar Three.js y GUI
npm install three lil-gui

# Iniciar servidor de desarrollo
npm run dev
```

El servidor de Vite se inicia en http://localhost:5173 con Hot Module Replacement (HMR) activo — los cambios se reflejan instantáneamente.

Laboratorio Paso 1 — Escena PBR y Sombras Suaves

ARCHIVO: MAIN.JS — ESTRUCTURA BASE

```
import * as THREE from 'three';
import GUI from 'lil-gui';

// 1. Escena y Render con Sombras Activas
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x05070f);

const camera = new THREE.PerspectiveCamera(60, window.innerWidth / window.innerHeight, 0.1, 100);
camera.position.set(0, 4, 7);
camera.lookAt(0, 1.5, 0);

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.shadowMap.enabled = true; // Activa sombras
renderer.shadowMap.type = THREE.PCFSoftShadowMap; // Suavizado PCF
document.body.appendChild(renderer.domElement);

// 2. Suelo que Recibe Sombras
const suelo = new THREE.Mesh(
 new THREE.PlaneGeometry(25, 25),
 new THREE.MeshStandardMaterial({ color: 0x111625, roughness: 0.5, metalness: 0.1 })
);
suelo.rotation.x = -Math.PI / 2;
suelo.receiveShadow = true;
scene.add(suelo);

// 3. Monolito PBR que Proyecta Sombras
const monolito = new THREE.Mesh(
 new THREE.CylinderGeometry(0.8, 1.1, 3, 32),
 new THREE.MeshPhysicalMaterial({
 color: 0x1e293b,
 roughness: 0.15,
 metalness: 0.1,
 clearcoat: 1.0,
 clearcoatRoughness: 0.1
 })
);
monolito.position.y = 1.5;
monolito.castShadow = true;
monolito.receiveShadow = true;
scene.add(monolito);
```

Laboratorio Paso 2 — Composición Lumínica Dual

ARCHIVO: MAIN.JS — CONTINUACIÓN: LUCES

```
// 1. Luz de Relleno Ambiental (Muy tenue)
const luzAmbiente = new THREE.AmbientLight(0xffffff, 0.2);
scene.add(luzAmbiente);

// 2. SpotLight Principal (Luz Clave Superior con Sombra)
const spotLuz = new THREE.SpotLight(0x38bdf8, 80);
spotLuz.position.set(5, 9, 5);
spotLuz.angle = Math.PI / 5;
spotLuz.penumbra = 0.5;
spotLuz.castShadow = true;
spotLuz.shadow.mapSize.width = 1024;
spotLuz.shadow.mapSize.height = 1024;
scene.add(spotLuz);

// 3. PointLight Cálida en la Cima (Emula Fuego/Antorcha)
const luzFuego = new THREE.PointLight(0xf97316, 25, 8);
luzFuego.position.set(0, 3.2, 0);
luzFuego.castShadow = true;
scene.add(luzFuego);
```

Análisis de la Composición

AmbientLight — 0.2

Relleno mínimo. Evita sombras negras absolutas sin aplanar el volumen.

SpotLight — Azul Frío

Color `0x38bdf8` (azul cielo). Intensidad 80. Shadow map 1024×1024 para sombras nítidas.

PointLight — Naranja Cálido

Color `0xf97316` (naranja fuego). Intensidad 25, radio 8 unidades. Contraste cálido/frío.

Laboratorio Paso 3 — Sistema de Fuego y Bucle a 60 FPS

ARCHIVO: MAIN.JS — CONTINUACIÓN: PARTÍCULAS Y ANIMACIÓN

```
// 1. Crear 3,000 Partículas en Memoria Continua
const total = 3000;
const coords = new Float32Array(total * 3);
const velY = new Float32Array(total);

for (let i = 0; i < total; i++) {
 coords[i * 3] = (Math.random() - 0.5) * 0.7; // X
 coords[i * 3 + 1] = 3.0 + Math.random() * 2.5; // Y
 coords[i * 3 + 2] = (Math.random() - 0.5) * 0.7; // Z
 velY[i] = 0.02 + Math.random() * 0.03; // Velocidad
}

const particulasGeo = new THREE.BufferGeometry();
particulasGeo.setAttribute('position', new THREE.BufferAttribute(coords, 3));

const particulasMat = new THREE.PointsMaterial({
 color: 0xff5500, size: 0.09, transparent: true,
 opacity: 0.85, blending: THREE.AdditiveBlending, depthWrite: false
});

const fuego = new THREE.Points(particulasGeo, particulasMat);
scene.add(fuego);

// 2. Bucle de Animación y Actualización de Vértices en GPU
const reloj = new THREE.Clock();
function animar() {
 requestAnimationFrame(animar);
 const t = reloj.getElapsedTime();

 // Parpadeo orgánico de la luz
 luzFuego.intensity = 20 + Math.sin(t * 12) * 6 + Math.random() * 4;

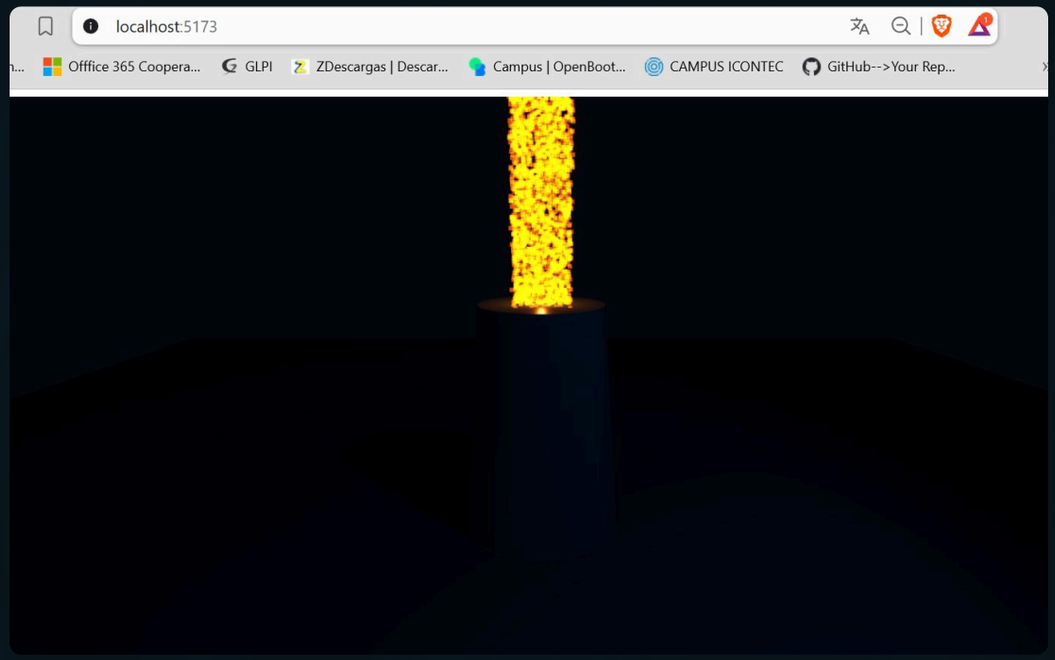
 // Movimiento ascendente de partículas
 const pos = particulasGeo.attributes.position.array;
 for (let i = 0; i < total; i++) {
 pos[i * 3 + 1] += velY[i]; // Sube en Y
 if (pos[i * 3 + 1] > 5.8) {
 pos[i * 3 + 1] = 3.0; // Reinicia en la base de la llama
 }
 }
 particulasGeo.attributes.position.needsUpdate = true; // Notificar a la GPU
 renderer.render(scene, camera);
}
animar();

// Ajuste responsivo de ventana (ADAPTACIÓN A CUALQUIER PANTALLA)
window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
```

En style.css

asegurarnos que el lienzo ocupe toda la pantalla, quitar margenes blancos, eliminar todo el codigo y colocar unicamente este codigo:

```
body {
  margin: 0;
  overflow: hidden;
  background-color: #05070f;
}
```



Desafío de Código en Vivo en Clase (30 Minutos)

Misión Práctica para los Estudiantes

Modificar en vivo el script del monolito con fuego para transformarlo en una escena de **"Lluvia Neón Cyberpunk"**.

*Listo muchachos: ya tenemos una antorcha con fuego ascendente, sombra suave y material barnizado. Ahora entramos al **Desafío en Vivo de 30 minutos**:*

1. *Invertir la física del bucle: que caiga desde arriba hacia el suelo como lluvia veloz.*
2. *Hacer que el suelo parezca asfalto mojado* (`roughness: 0.05`, `metalness: 0.85`).
3. *Cambiar la paleta a Cyan Neón* (`0x00ffff`).

🔽 Invertir la Gravedad

Cambiar el bucle para que las partículas **caigan desde Y = 10 hasta el suelo (Y = 0)** a mayor velocidad. Invertir el signo de `velY` y ajustar los límites de reinicio.

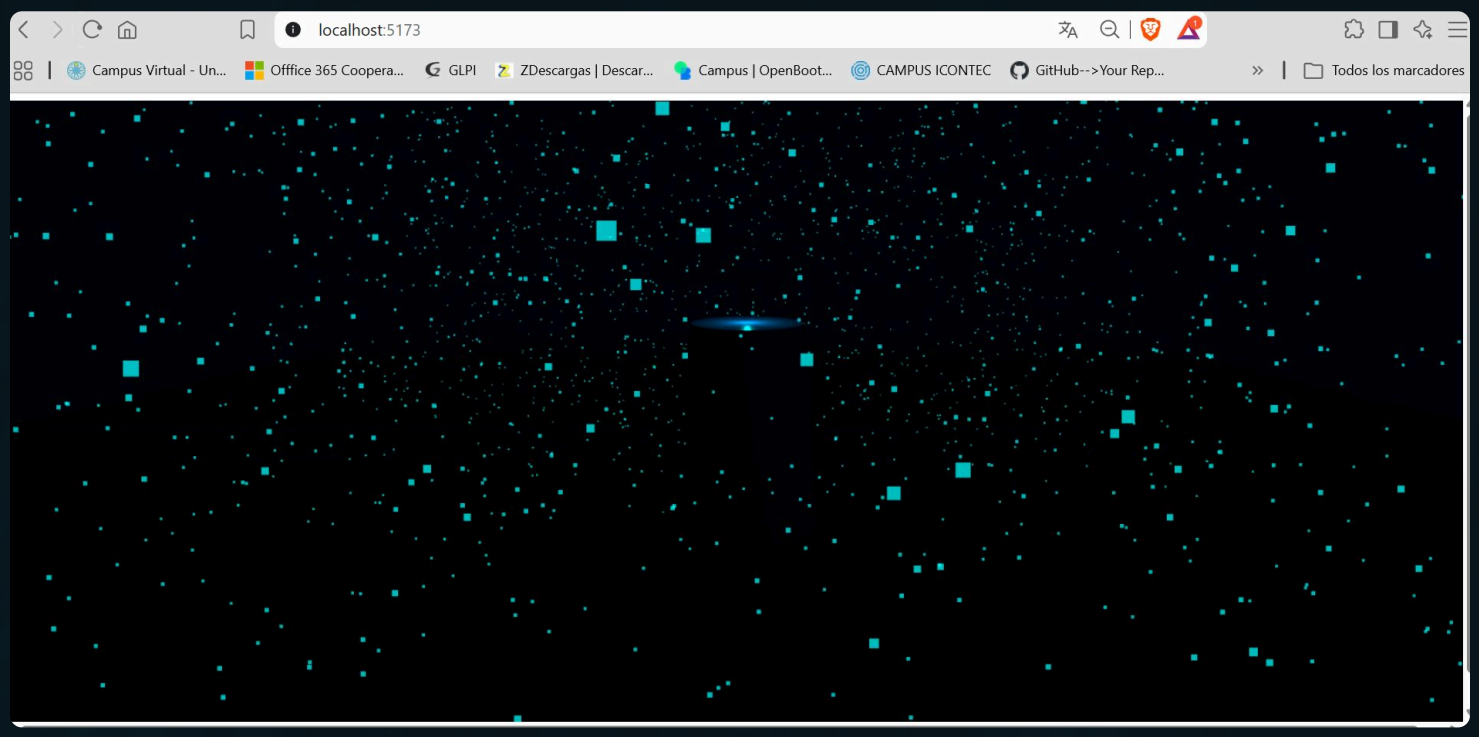
💧 Efecto Charco Mojado

Modificar el material del suelo a `roughness: 0.05` y `metalness: 0.85` para que refleje el entorno como **asfalto mojado** bajo la lluvia neón.

🌈 Paleta Neón

Cambiar el color de las partículas a **Cyan Neón (**

resultado esperado o similar:



Puntos clave para el resultado esperado:

1. **La dirección de movimiento:** En el bucle de animación, en vez de sumar (`pos[i * 3 + 1] += velY[i]`), ahora deben restar (`-= velY[i]`) para que caiga.
2. **El suelo reflectante:** Destácales cómo al bajar `roughness` a `0.05` y subir `metalness` a `0.85`, el suelo pasa de parecer tierra mate a asfalto empapado que refleja las luces neón.
3. **El color cian neón:** El cambio a `0x00ffff` en el material y la luz produce ese contraste característico con la niebla oscura.

Resumen del Ejercicio Realizado y Conclusiones de la Jornada

Resumen del Ejercicio Realizado en Clase

- Construimos una escena fotorrealista combinando **materiales físicos PBR con barniz (clearcoat)**.
- Diseñamos un esquema lumínico coordinado con **sombras suaves (PCFSoftShadowMap)**.
- Implementamos una simulación de fuego con **3,000 partículas en un único Draw Call**, animadas en tiempo real sin pérdida de rendimiento.

Conclusiones Clave que Deben Llevarse Hoy

- **PBR es estándar:** La rugosidad y metalicidad reemplazan las texturas planas por respuestas ópticas naturales.
- **Eficiencia en GPU:** Miles de elementos móviles no se crean como mallas separadas, sino como vértices en un `BufferGeometry` con `needsUpdate = true`.
- **Equilibrio de Sombras:** Las sombras añaden profundidad realista pero son costosas; deben reservarse para las luces clave.

🚀 **Próxima Semana:** ¡Cerramos el Momento 1 y entramos al Momento 2 con Motores de Físicas 3D y Detección de Colisiones!